🚀 Collections Framework Masterclass

Chapter 4 — ArrayList (Complete Internal Working)

> **Interview Importance:** ⭐⭐⭐⭐⭐

If there is **one Collection class** you must master for Java interviews, it is **ArrayList**.

Interviewers don't just ask how to use `ArrayList`; they expect you to understand **how it works internally**, **how it grows**, **why operations have different time complexities**, and **when it should or should not be used**.

What is ArrayList?

`ArrayList` is a **resizable array implementation** of the `List` interface.

Package

```
java.util.ArrayList
```

Declaration

```
public class ArrayList<E>
        extends AbstractList<E>
        implements List<E>,
                   RandomAccess,
                   Cloneable,
                   Serializable
```

Where Does ArrayList Fit?

```
Iterable
    ↑
Collection
    ↑
```

```
List
    ↑
ArrayList
```

Characteristics of ArrayList

✅ Maintains insertion order

✅ Allows duplicate elements

✅ Allows multiple null values

✅ Index-based access

✅ Dynamically grows

❌ Not synchronized

❌ Not thread-safe

Internal Structure

Many beginners think ArrayList is some magical data structure.

It is not.

Internally it is simply an **array of objects**.

```
Index

0     1     2     3     4

↓

+-----+-----+-----+-----+-----+
| A   | B   | C   | D   | E   |
+-----+-----+-----+-----+-----+
```

Internally Java stores elements in

```
transient Object[] elementData;
```

This single array is the heart of ArrayList.

Creating an ArrayList

```
ArrayList<String> list = new ArrayList<>();
```

What actually happens?

Java creates an empty ArrayList.

Important:

No large array is allocated immediately.

Default Constructor

```
ArrayList<String> list =
        new ArrayList<>();
```

Internally

```
elementData = DEFAULTCAPACITY_EMPTY_ELEMENTDATA;
```

Initially

```
Size = 0

Capacity = 0
```

The internal array is shared and empty.

What Happens on First add()?

```
list.add("Java");
```

Now Java creates the first internal array.

Default capacity becomes

```
10
```

Internal structure

```
Capacity = 10

+------+------+------+------+------+------+------+------+------+------+
|Java |null |null |null |null |null |null |null |null |null |
+------+------+------+------+------+------+------+------+------+------+
```

Size vs Capacity

This is one of the favorite interview questions.

Size

Number of actual elements.

```
ArrayList<Integer> list =
new ArrayList<>();

list.add(10);
list.add(20);
```

Size

```
2
```

Capacity

Total number of elements that can be stored before resizing.

Current capacity

```
10
```

Interview Answer

```
Size

↓

Actual elements

Capacity

↓

Available storage
```

Capacity Example

```
ArrayList<Integer> list =
new ArrayList<>();
```

After first insertion

```
Capacity = 10

Size = 1
```

After adding

```
10 elements
```

Capacity

```
10
```

After adding

```
11th element
```

Capacity grows.

Growth Algorithm

Interview Favorite ⭐⭐⭐⭐⭐

Suppose

Current Capacity

```
10
```

Need one more element.

Java creates

```
New Capacity

15
```

Formula

```
Old Capacity + (Old Capacity >> 1)
```

Meaning

```
Old + 50%
```

Example

```
10 → 15

15 → 22

22 → 33

33 → 49

49 → 73

73 → 109
```

Growth is approximately **1.5×**, not double.

Why Not Double the Size?

Imagine storing

```
10 million objects
```

Doubling every resize would waste large amounts of memory.

Using **1.5× growth** balances

- Memory usage
- Performance
- Fewer reallocations

What Happens During Resize?

Suppose

```
Capacity = 10
```

You insert the 11th element.

Old array

```
+----+----+----+----+----+----+----+----+----+----+
|1|2|3|4|5|6|7|8|9|10|
+----+----+----+----+----+----+----+----+----+----+
```

Java creates a new larger array.

```
Capacity = 15
```

Copies all existing elements.

Then inserts the new element.

Old array becomes eligible for Garbage Collection.

Resize Cost

This copy operation is expensive.

Because

```
All elements

↓

Copied again
```

Complexity

```
O(n)
```

Why add() is O(1)?

Interview Favorite ⭐⭐⭐⭐⭐

Most insertions occur at the end.

Example

```
list.add(100);
```

Simply place element in the next free slot.

```
O(1)
```

Only during resizing

```
O(n)
```

Therefore

Average complexity

```
Amortized O(1)
```

Amortized Time Complexity

Interview Question ⭐⭐⭐⭐⭐

What is Amortized O(1)?

Answer

Most operations are O(1).

Occasionally a resize costs O(n).

When averaged across many insertions, each `add()` is effectively O(1).

---

ensureCapacity()

Sometimes we already know how many elements we need.

Example

```
ArrayList<Employee> list =
new ArrayList<>();

list.ensureCapacity(10000);
```

Benefits

- Reduces resizing
- Improves performance
- Fewer array copies

Useful when loading large datasets from files or databases.

---

trimToSize()

Suppose

Capacity

```
1000
```

Size

```
50
```

Extra memory is wasted.

Use

```
list.trimToSize();
```

Now

```
Capacity = 50
```

Memory optimized.

Accessing Elements

```
list.get(5);
```

How?

Because arrays support direct indexing.

Formula

```
Address + (Index × Element Size)
```

Therefore

```
O(1)
```

Random access is extremely fast.

Why ArrayList Implements RandomAccess?

Notice

```
implements RandomAccess
```

`RandomAccess` is a **marker interface**.

It contains **no methods**.

It tells Java

> This collection supports fast random access.

Algorithms can optimize based on this information.

Insertion

Adding at end

```
list.add(100);
```

Complexity

```
O(1)
```

Adding at beginning

```
list.add(0,100);
```

Current array

```
A B C D
```

Need

```
100 A B C D
```

Every element shifts one position.

Complexity

```
O(n)
```

Removal

Removing last element

```
list.remove(list.size()-1);
```

Complexity

```
O(1)
```

Removing first element

```
list.remove(0);
```

Remaining elements shift left.

```
A B C D

↓

Remove A

↓

B C D
```

Complexity

```
O(n)
```

---

contains()

Example

```
list.contains("Java");
```

Java searches sequentially.

```
Java?

↓

Element 1

↓

Element 2
```

```
↓

Element 3
```

Complexity

```
O(n)
```

Iteration

```
for(String s : list)
```

Complexity

```
O(n)
```

Fast because memory is contiguous.

This makes ArrayList more CPU cache-friendly than LinkedList.

Null Values

ArrayList allows multiple null values.

```
list.add(null);
list.add(null);
```

Perfectly valid.

Duplicate Elements

Duplicates are allowed.

```
list.add("Java");
list.add("Java");
```

Output

```
[Java, Java]
```

Thread Safety

Interview Favorite ⭐⭐⭐⭐⭐

Is ArrayList thread-safe?

No.

Multiple threads modifying the same ArrayList may lead to

- Data corruption
- Lost updates
- Unexpected exceptions

How to Make It Thread-Safe?

Option 1

```
List<String> list =
Collections.synchronizedList(
new ArrayList<>()
);
```

Option 2

```
CopyOnWriteArrayList
```

We'll study this later.

ArrayList vs Array

Array	ArrayList
Fixed Size	Dynamic
Stores primitives & objects	Stores objects (Generics use wrapper types for primitives)
Faster	Slightly slower due to abstraction
No rich API	Rich Collection API

ArrayList vs LinkedList

Feature	ArrayList	LinkedList
Internal Structure	Dynamic Array	Doubly Linked List
Random Access	O(1)	O(n)
Insert at End	O(1)*	O(1)
Insert at Beginning	O(n)	O(1) (after locating node)
Remove by Index	O(n)	O(n)
Memory Usage	Lower	Higher (node references)
Cache Locality	Excellent	Poor

*Amortized O(1)

Common Mistakes

Mistake 1

```
ArrayList list =
new ArrayList();
```

Prefer Generics

```
ArrayList<String> list =
new ArrayList<>();
```

Mistake 2

Creating a huge ArrayList without pre-sizing.

```
new ArrayList<>();
```

Better

```
new ArrayList<>(100000);
```

or

```
list.ensureCapacity(100000);
```

Mistake 3

Using ArrayList for frequent insertions at the beginning.

Better choice

```
LinkedList
```

Internal Source Code (Simplified)

```
public boolean add(E e) {

    ensureCapacityInternal(size + 1);

    elementData[size++] = e;
```

```
    return true;
}
```

Resize happens only if required.

Otherwise, the element is simply placed into the next free slot.

Time Complexity

Operation	Complexity
get(index)	O(1)
set(index)	O(1)
add(end)	O(1) Amortized
add(index)	O(n)
remove(last)	O(1)
remove(index)	O(n)
contains()	O(n)
indexOf()	O(n)
iteration	O(n)
clear()	O(n)

Frequently Asked Interview Questions

Q1. How does ArrayList work internally?

It stores elements in a dynamically resizable `Object[]` array.

Q2. Difference between size and capacity?

- **Size** = Number of actual elements.
- **Capacity** = Size of the internal array.

Q3. What is the default capacity?

With the default constructor, the internal array is initially empty. On the **first insertion**, the capacity grows to **10**.

Q4. What is the growth formula?

Approximately:

```
newCapacity = oldCapacity + (oldCapacity >> 1)
```

That is roughly **1.5×** the old capacity.

Q5. Why is add() O(1)?

Most insertions simply place the element into the next available slot. Only occasional resizing requires copying elements, making the average cost **amortized O(1)**.

Q6. Is ArrayList synchronized?

No.

Q7. Why does ArrayList implement RandomAccess?

To indicate that it supports efficient random (index-based) access, allowing algorithms to optimize accordingly.

Q8. When should you use ensureCapacity()?

When you know approximately how many elements you'll store, to reduce resizing and improve performance.

Q9. What does trimToSize() do?

It shrinks the internal capacity to match the current size, reducing unused memory.

Key Takeaways

- `ArrayList` is backed by a dynamically resizable `Object[]`.
- It provides O(1) random access through array indexing.
- Capacity and size are different concepts.
- Capacity grows by approximately **1.5×** when required.
- `add()` is **amortized O(1)** because resizing happens infrequently.
- `ensureCapacity()` improves bulk insertion performance.
- `trimToSize()` helps reclaim unused memory.
- `ArrayList` is not thread-safe.

Next Chapter

🚀 Chapter 5 – LinkedList (Complete Internal Working)

We'll cover:

- Doubly Linked List Architecture
- Node Structure
- Head & Tail References
- Internal Source Code
- Memory Layout
- Why `get()` is O(n)
- Why Insertion is Fast
- Queue & Deque Operations
- ArrayList vs LinkedList (Deep Comparison)
- Performance Analysis
- 20+ Interview Questions